

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

BINGAMALLA HANVITHA (VTU25643)

*10211CS224
Full Stack Application Development
Slot : E2*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "SMART CAMPUS EVENT MANAGEMENT SYSTEM" by BINGAMALLA HANVITHA (VTU24829) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Associate Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra. M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

BINGAMALLA HANVITHA

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a web-based application developed to simplify and automate the management of campus events, workshops, and seminars in educational institutions. Traditional event management methods are often manual, time-consuming, and prone to errors in handling registrations, updates, and communication.

This system provides a centralized platform where students can securely register, log in, browse upcoming events, enroll in events, and provide feedback. Administrators are provided with a dedicated dashboard to create, update, delete, and monitor events efficiently. The system also includes features such as event scheduling, registration deadline management, department-based filtering, announcements, and email notifications.

By integrating frontend technologies like HTML, CSS, and Thymeleaf with backend technologies such as Spring Boot, Spring Data JPA, and MySQL, the system ensures scalability, security, and efficient data handling. Overall, the application reduces manual effort, improves communication between students and administrators, and enhances the overall efficiency of campus event management.

Keywords: Smart Campus, Event Management, Spring Boot, Thymeleaf, MySQL, Automation.

LIST OF FIGURES

1.1	Framework	3
3.1	Execution Procedure	9
4.1	Project Architecture	13
4.2	Data Flow Diagram	14
5.1	Register	18
5.2	Admin	19
5.3	Home	20
5.4	Add	21
5.5	Otp Notification	21
5.6	Event Management	22
5.7	Email Verification	23
5.8	Registration Success	24

LIST OF TABLES

2.1	Comparative Analysis of Market Applications	5
5.1	Test Cases for Event Management System	17
7.1	Mapping of Project to Complex Engineering Problem (CEP)	26
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	27
7.3	SDG Mapping for the Project	28

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
EMS	Event Management System
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
MVC	Model View Controller
OTP	One Time Password
SQL	Structured Query Language
UI	User Interface

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.2 Backend Implementation	7
3.3 Database Implementation	10
4 METHODOLOGIES	12
4.1 Project Architecture	12

4.2	DataFlow Diagram	13
4.3	Module 1:	14
4.4	Module 2:	14
4.5	Module 3:	15
5	RESULTS AND DISCUSSIONS	16
5.1	System Modules and Features	16
5.2	System Testing and Validation	17
5.3	System Interface Visualizations	18
5.3.1	Register Event Page	18
5.3.2	Admin Login Page	19
5.3.3	Home Page	20
5.3.4	Add Event Page	20
5.3.5	Otp Notification	21
5.3.6	Event Management Page	22
5.3.7	Email Verification Page	23
5.3.8	Registration Success Page	24
6	CONCLUSION AND FUTURE ENHANCEMENTS	25
6.1	Conclusion	25
6.2	Future Enhancements	25
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	26
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	26
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	27
7.3	SDG Mapping	28
	References	28

Chapter 1

INTRODUCTION

1.1 Introduction

The Smart Campus Event Management System is a web-based application developed to simplify and automate the management of campus events, workshops, and seminars in educational institutions. Traditional event management processes are often handled manually, which can lead to inefficiencies, communication delays, and errors in registration handling.

This system provides a centralized platform where students can securely register, log in, browse upcoming events, register for events, and provide feedback. Administrators can efficiently manage the complete event lifecycle by creating, updating, deleting, and monitoring events through a dedicated admin dashboard.

1.2 Aim of the Project

The aim of this project is to develop a web-based system for managing campus events efficiently. The system focuses on automating event creation, event registration, and user authentication to reduce manual effort and improve communication between administrators and students.

1.3 Scope of the Project

The scope of this project includes managing campus events through a centralized web platform. Students can register, view, and participate in events, while administrators can manage event creation, updates, and registrations.[5] The system can be further enhanced with features such as mobile application support, cloud deployment, and advanced analytics for better performance and scalability.

1.4 Framework

As illustrated in figure 1.1, The Smart Campus Event Management System is developed using a multi-tier architecture based on the Spring Boot framework.[4] The system follows the Model-View-Controller (MVC) design pattern, which ensures modularity, scalability, and maintainability. The frontend is built using HTML5, CSS3, and Thymeleaf for creating dynamic and user-friendly web pages. The backend uses Spring MVC, Spring Data JPA, and Spring Security to handle business logic, database operations, and secure authentication. The database layer is managed using MySQL, which stores user details, event information, registrations, and feedback.

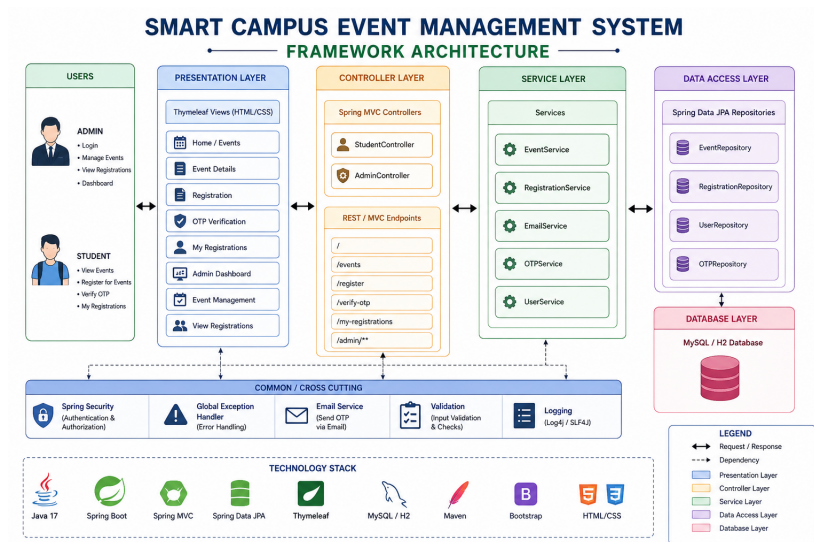


Figure 1.1: Framework

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management applications are widely used in the market, such as Eventbrite, BookMyShow, Townscript, and Meetup.[3] These platforms provide important features like event creation, ticket booking, participant registration, and notification services, which help organizers manage events efficiently and improve the overall user experience.

Applications like Eventbrite and Townscript allow organizers to create and manage events, handle participant registrations, and communicate with attendees through notifications and updates. These platforms provide a structured system for event planning and participant management.

BookMyShow mainly focuses on ticket booking for entertainment events such as movies, concerts, and public shows. It provides users with easy ticket booking facilities and event discovery features. Meetup is another platform that helps communities and groups organize events and connect with participants. Most existing applications are designed for large-scale public events and often include advanced features such as online payment integration, digital ticket generation, seat booking, and event analytics. These features are useful for commercial and public event management.

However, these platforms may not be suitable for internal campus event management, where simple registration, OTP verification, participant management, and direct communication are more important than commercial features. Many of these systems also include complex functionalities that may not be necessary for college-level event management.

The Smart Campus Event Management System is designed specifically for educational institutions to manage campus events in a simple and efficient manner. It provides features such as user registration, login, event creation, event deletion, OTP-based event registration, email notifications, participant management, and mobile accessibility.

Compared to existing market applications, the proposed system focuses on simplicity, secure authentication, and efficient management of internal campus events. The following table shows the comparative analysis of existing applications and the proposed system based on various features.

Table 2.1: Comparative Analysis of Market Applications

Feature	Eventbrite	BookMyShow	Townscript	Proposed System
Event Creation	Yes	No	Yes	Yes
User Registration/Login	Yes	Yes	Yes	Yes
Event Registration	Yes	Yes	Yes	Yes
OTP Verification	No	No	No	Yes
Email Notification	Yes	Yes	Yes	Yes
Participant Management	Yes	Limited	Yes	Yes
Event Deletion	Yes	No	Yes	Yes
Mobile Access	Yes	Yes	Yes	Yes
Campus Event Support	Limited	No	Limited	Yes
Admin Control	Yes	Limited	Yes	Yes
Event Status Tracking	No	No	Limited	Yes

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the application is developed using HTML, CSS, and basic JavaScript. The project is managed using the Spring Boot framework, where the frontend templates are placed in the `templates` folder. The development is done using Eclipse IDE, and the application is tested using a web browser.

3.1.2 Structure of the Project The project follows a structured organization to improve maintainability and proper separation of functionalities:

- **/src/main/java/com.example.event/controller:** Contains controller classes such as `EventController.java` to handle user requests, page navigation, event management, registration, login, and OTP verification.
- **/src/main/resources/templates:** Contains HTML files such as `login.html`, `signup.html`, `index.html`, `add.html`, `register.html`, `verifyOtp.html`, and `registrations.html` for user interface.
- **/src/main/resources/static:** Contains static resources such as CSS files and images used in the application.

3.1.3 Execution Procedure

The application is executed by running the main Spring Boot application file in Eclipse IDE. Once the server starts successfully, the application can be accessed through a web browser using:

`http://localhost:8080`

After opening the application, users can perform operations such as signup, login, event creation, and event registration. OTP verification is used during event registration for security, and email notifications are sent after successful registration. The frontend interface provides easy navigation for users to perform operations such as viewing events, registering for events, and verifying registration through OTP.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Smart Campus Event Management System is developed using Java and Spring Boot framework. The development environment is set up using Eclipse IDE with Java Development Kit (JDK) and Maven for dependency management. Spring Boot dependencies are configured through the pom.xml file.

MySQL database is configured for storing user details, event information, and registration records. The database connection, server configuration, and mail configuration are maintained in the application.properties file. The backend environment supports OTP verification, email notification, and event management operations efficiently. The system provides a simple interface for users and administrators to manage events efficiently.

3.2.2 Structure of the Project

The backend of the Smart Campus Event Management System follows a structured project organization for better code management and maintainability. The project is divided into different packages such as controller, service, repository, and model to handle various functionalities efficiently. The backend of the project is organized into different packages to maintain proper code structure and manage application functionalities efficiently.

- **/src/main/java/com.example.event/controller:** Contains controller classes such as `EventController.java` to handle user requests, page navigation, event management, registration, login, and OTP verification.
- **/src/main/java/com.example.event/service:** Contains service classes such as `EventService.java`, `RegistrationService.java`, `UserService.java`, and `EmailService.java` to handle business logic, OTP generation, email notifications, and event operations. This layer acts as a bridge between the controller and repository layers, ensuring proper execution of application functionalities.
- **/src/main/java/com.example.event/repository:** Contains repository interfaces such as `EventRepository.java`, `RegistrationRepository.java`, and `UserRepository.java` for database operations using Spring Data JPA.
- **/src/main/java/com.example.event/model:** Contains entity classes such as `Event.java`, `Registration.java`, and `User.java` to represent database tables.
- **application.properties:** Contains database configuration, server configuration, and mail configuration settings.
- **EventManagerApplication.java:** Main application file used to start and run the Spring Boot application.

3.2.3 Execution Procedure

As illustrated in figure 3.1, The backend execution of the Smart Campus Event Management System begins when the Spring Boot application is started. The embedded server initializes and loads all necessary configurations and dependencies. User requests from the frontend are received by the controller layer, which processes and forwards them to the service layer. The service layer handles the business logic such as validation, event processing, and user management. It then interacts with the repository layer, which performs CRUD operations on the MySQL database using Spring Data JPA. After processing, the response is sent back through the controller to the frontend. Security is maintained using Spring Security, and email notifications are handled using JavaMailSender.

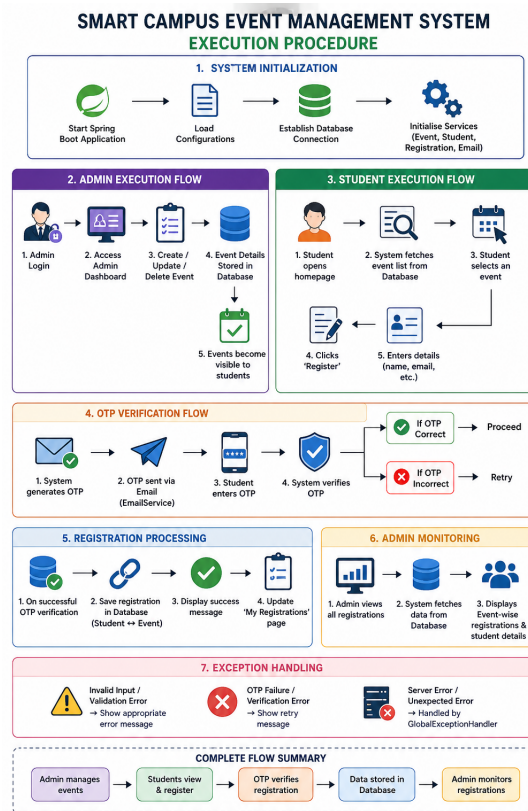


Figure 3.1: Execution Procedure

3.3 Database Implementation

The application uses MySQL database for storing user details, event information, and registration records. The database connection is configured in the application.properties file by specifying the database URL, username, and password. Spring Data JPA is used for database connectivity and operations such as insert, update, delete, and retrieval. Hibernate is used for automatic table creation and updating. This ensures efficient data management in the application. The database configuration provides secure storage and easy retrieval of application data for efficient system performance.

3.1.2 Schema Structure

The database schema of the Smart Campus Event Management System is designed to store and manage application data in an organized manner. It consists of three main tables: Event, Registration, and User. Each table contains specific fields related to event details, participant registration, and user authentication. The schema structure helps in maintaining proper data organization and ensures easy retrieval and management of information.

3.1.3 Tables created

The database consists of three main tables used for storing and managing application data efficiently. Each table is designed to store specific information related to the system.

Event Table: Stores event details such as event name, date, location, and status. It helps in managing and displaying event information.

Registration Table: Stores participant registration details such as student name, event name, date, time, and email. It is used to maintain records of students registered for events.

User Table: Stores user account details such as username, email, and password

for login and authentication. It ensures secure access to the system.

These tables are created automatically using Hibernate through Spring Data JPA configuration. The table structure supports easy insertion, deletion, updating, and retrieval of data. This improves the efficiency of data management and ensures proper functioning of the application.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

As illustrated in figure 4.1, The Smart Campus Event Management System follows the Model-View-Controller (MVC) architecture for better organization and maintainability. The frontend layer handles user interaction through web pages, while the controller processes user requests and manages application flow. The service layer contains business logic, and the repository layer performs database operations using Spring Data JPA. The MySQL database stores user, event, and registration details, ensuring efficient data management.

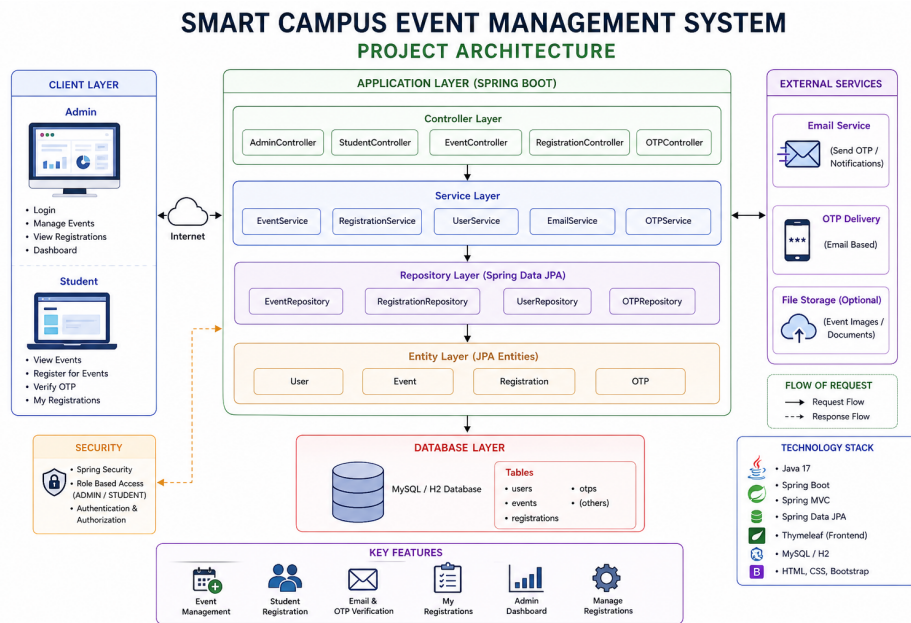


Figure 4.1: Project Architecture

4.2 DataFlow Diagram

As illustrated in figure 4.2, The Data Flow Diagram (DFD) represents how data moves within the system. It shows the interaction between the user, frontend, back-end components, and the database. When a user performs an action, the request is sent through the web browser to the controller. The controller forwards the request to the service layer, where the business logic is applied.

The processed data is then sent back through the same layers and displayed to the user. This flow ensures proper data handling and smooth communication between different components of the system.

SMART CAMPUS EVENT MANAGEMENT SYSTEM DATA FLOW DIAGRAM (DFD)

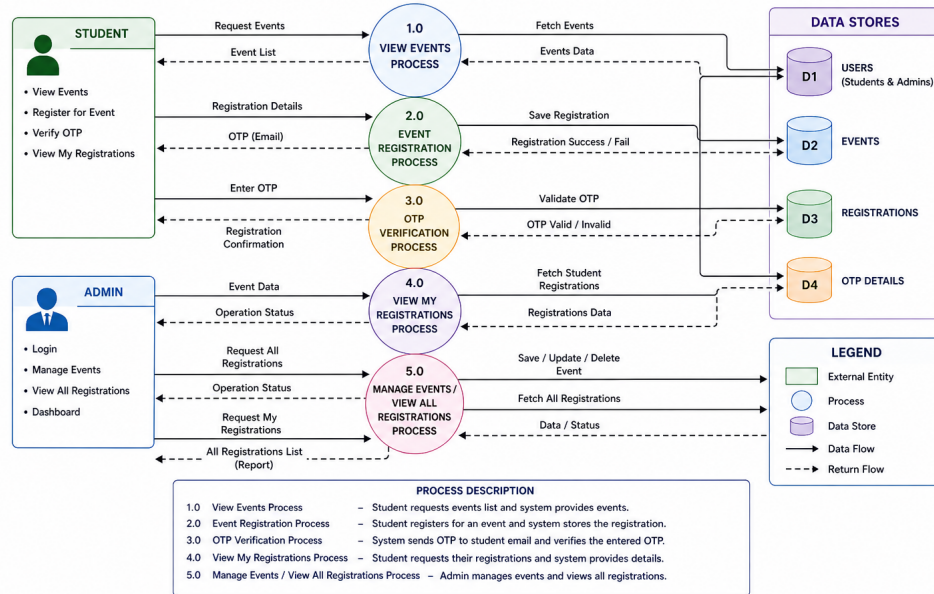


Figure 4.2: Data Flow Diagram

4.3 Module 1:

User Management Module: This module manages user registration, login, and authentication. It allows users to create accounts and securely access the system using their credentials. The module ensures that only authorized users can perform operations and helps maintain user information such as name and email.

4.4 Module 2:

Event Management Module: This module manages event creation, viewing, and deletion. It allows administrators to add event details such as event name, date, location, and status. The module helps in organizing and maintaining event information efficiently.

4.5 Module 3:

Registration Module: This module manages participant registration for events. It verifies registration through OTP and stores participant details such as name, event, date, and email. The module also sends email notifications after successful registration.

Advantages of this Project

a) Easy to Use: The system provides a simple and interactive interface for users. Even beginners can easily understand and use the application without difficulty.

b) Time Saving: Automation reduces the time required for managing events manually. Tasks such as adding, updating, and viewing events are completed quickly.

Disadvantages

a) Internet Dependency: The system requires an active internet connection to function properly. Without connectivity, users cannot access the application.

b) Server Issues: If the server is down or running slowly, the performance of the application may be affected, causing delays in processing requests.

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management System was successfully designed and implemented. The application provides a simple and efficient platform for managing campus events. The developed system supports user registration, login, event creation, event registration, OTP verification, and email notification functionalities.

5.1 System Modules and Features

[1] The Smart Campus Event Management System is divided into different modules to perform various operations efficiently. The User Management Module handles user registration, login, and authentication for secure system access. The Event Management Module manages event creation, viewing, and deletion of event details.

The Registration Management Module handles participant registration for events and stores registration details in the database. OTP verification is used to validate event registrations securely. The Email Notification feature sends OTP and confirmation messages to users after successful registration.

These modules work together to provide efficient event management, secure access, and better communication between administrators and participants.

5.2 System Testing and Validation

Test Case ID	Test Case Description	Input	Expected Result
TC-01	Add New Event	Event Name: Birthday Party, Date: 20-05-2026, Location: Chennai	Event is successfully added and displayed in the event list.
TC-02	Add Event with Empty Fields	Event Name: (empty), Date: (empty), Location: (empty)	System shows an error message asking to fill all required fields.
TC-03	Valid User Registration	Name: Sravanthi, Username: srav, Password: 1234	User is successfully registered and redirected to login page.
TC-04	Valid Login	Username: srav, Password: 1234	User is successfully logged in and redirected to home page.
TC-05	Invalid Login	Username: srav, Password: wrongpass	System displays “Invalid username or password” message.
TC-06	Register for Event	Student Name: Sravanthi, Event: Birthday Party	Registration is successfully saved and displayed in registrations list.
TC-07	View Registrations	Click on “View Registrations” page	All registered participants are displayed correctly.
TC-08	Delete Event	Click delete button for an event	Selected event is removed from the list successfully.

Table 5.1: Test Cases for Event Management System

5.3 System Interface Visualizations

5.3.1 Register Event Page

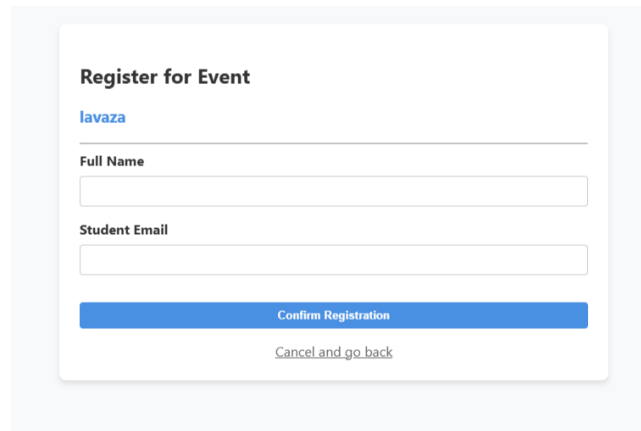
A screenshot of a web form titled "Register for Event". The form is white with a light gray border. It contains a blue link "lavaza" above a horizontal line. Below the line are two input fields: "Full Name" and "Student Email". At the bottom, there is a blue button labeled "Confirm Registration" and a link labeled "Cancel and go back" below it.

Figure 5.1: Register

Fig 5.1 shows The page consists of input fields including Full Name, Email Address, and Student ID. These fields are used to uniquely identify the participant and ensure proper communication. The system performs validation to check whether all fields are correctly filled and whether the email format is valid.

After entering the required details, the user clicks on the **Send OTP** button. The system then generates a One-Time Password (OTP) and sends it to the registered email address for verification. This step enhances security and ensures that only valid users can register for the event.

The **Cancel** button allows users to clear the form or exit the registration process. Once the OTP is verified successfully, the registration is confirmed and stored in the database. This page ensures secure, fast, and efficient event registration while maintaining data accuracy and user authentication.

5.3.2 Admin Login Page

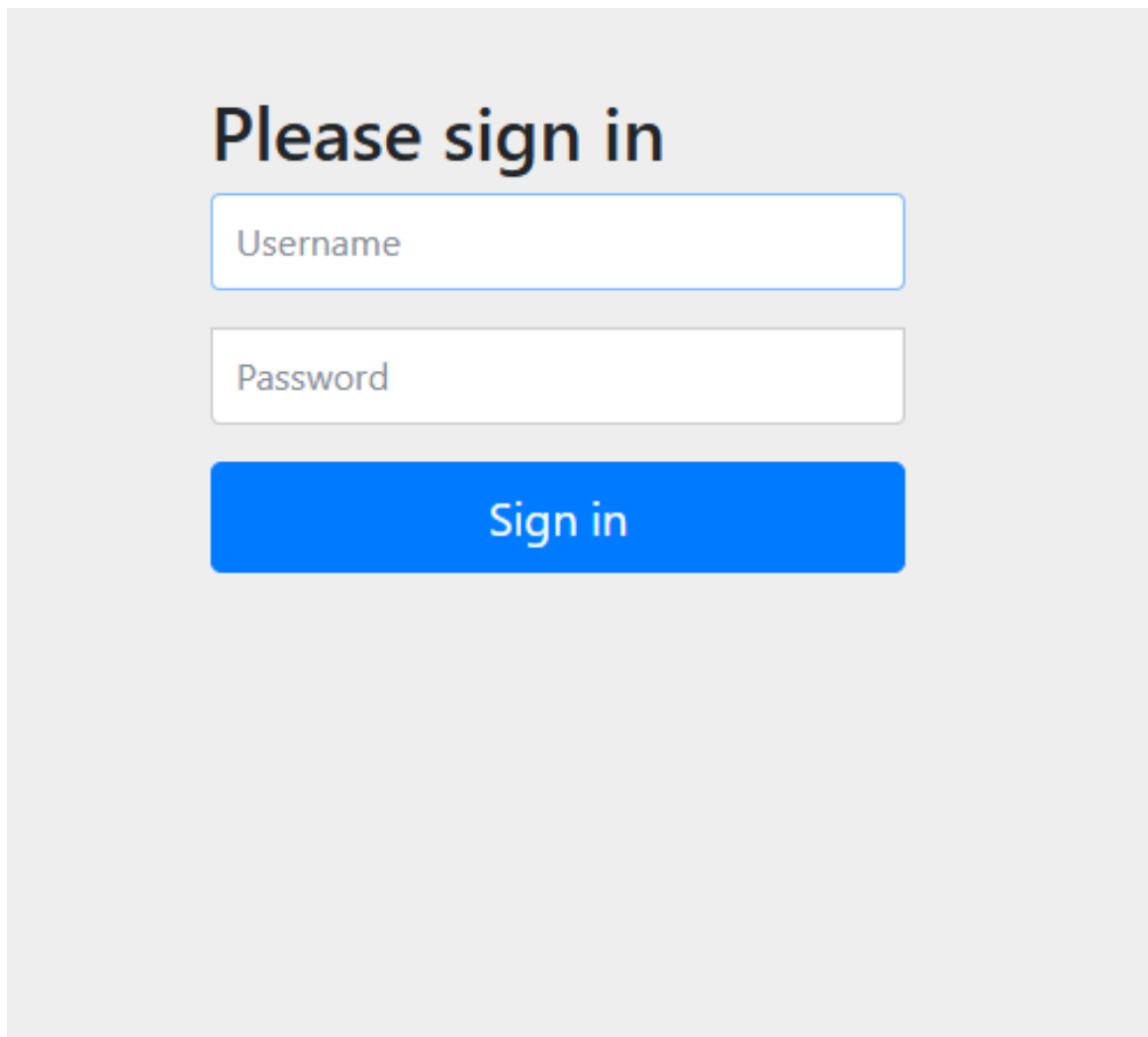
The image shows a login interface on a light gray background. At the top, the text "Please sign in" is displayed in a large, bold, black font. Below this text are two input fields: the first is labeled "Username" and the second is labeled "Password". Both fields are white with a thin blue border. Below the password field is a blue rectangular button with the text "Sign in" in white. The entire form is centered on the page.

Figure 5.2: Admin

From the Fig 5.2, The page contains input fields for **Username** and **Password**. The administrator is required to enter valid login credentials to access the system. Proper validation is implemented to ensure that both fields are not empty and that the entered details match the records stored in the database. If invalid credentials are entered, an error message is displayed, prompting the user to re-enter the correct details. This ensures that unauthorized users cannot gain access to the system.

5.3.3 Home Page

The screenshot shows a web interface titled "Upcoming Events". At the top, there are two input fields for filtering: "Department" (with the placeholder "e.g. Computer Science") and "Event Type" (with the placeholder "e.g. Seminar"). To the right of these fields are two buttons: "Filter" (blue) and "Clear" (orange). Below the filter section, there are two event cards. The first card is for an event named "tantraz". It lists the date as "13 May 2026, 10:30", the location as "33 block", the department as "Computer Science" (in a light blue pill), and the type as "Seminar" (in a light grey pill). Below this information, it says "tantraz on may 13" and provides two buttons: "View Details" (orange) and "Register" (green). The second card is for an event named "lavaza". It lists the date as "15 May 2026, 11:40", the location as "reasearch park", the department as "Computer Science" (in a light blue pill), and the type as "Seminar" (in a light grey pill). Below this information, it says "lavaza on may 15" and provides two buttons: "View Details" (orange) and "Register" (green).

Figure 5.3: Home

From the Fig 5.3, the Home page displays the list of upcoming events available in the Smart Campus Event Management System. It allows users to view event details such as event name, date, location, and status, and provides options to manage events.

5.3.4 Add Event Page

From the below Fig 5.4, the Add Event page allows users to create and add new event details in the Smart Campus Event Management System. It collects information such as event name, date, location, and status, which are stored in the database for event management.

Upcoming Events

Department

Event Type

Filter

Clear

tantraz

Date: 13 May 2026, 10:30

Location: 33 block

Department: Computer Science

Type: Seminar

tantraz on may 13

View Details Register

lavaza

Date: 15 May 2026, 11:40

Location: reasearch park

Department: Computer Science

Type: Seminar

lavaza on may 15

View Details Register

Figure 5.4: Add

5.3.5 Otp Notification

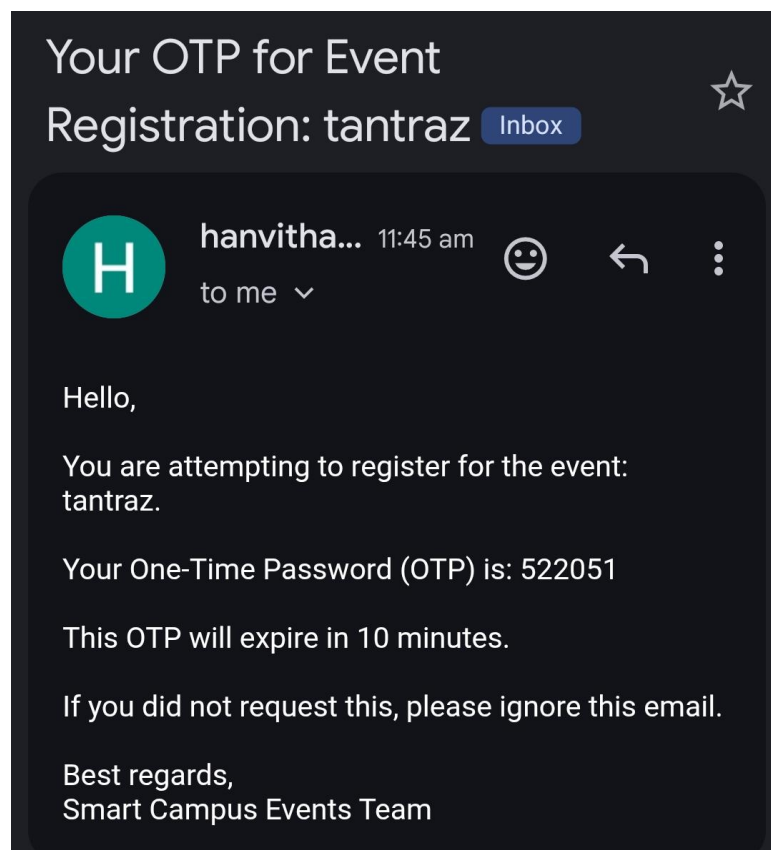
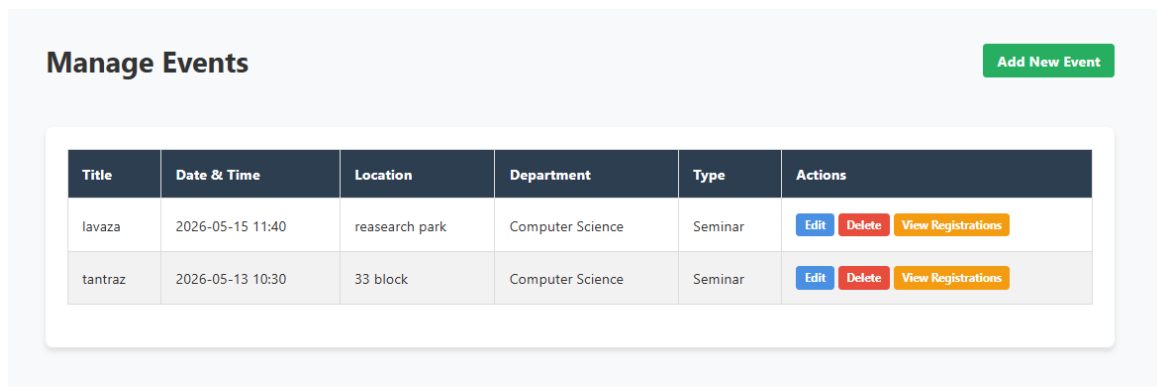


Figure 5.5: Otp Notification

From the Fig 5.5, the OTP Verification email shows the OTP sent to the user for validating event registration. It helps in verifying the user and ensures secure completion of the registration process.

5.3.6 Event Management Page



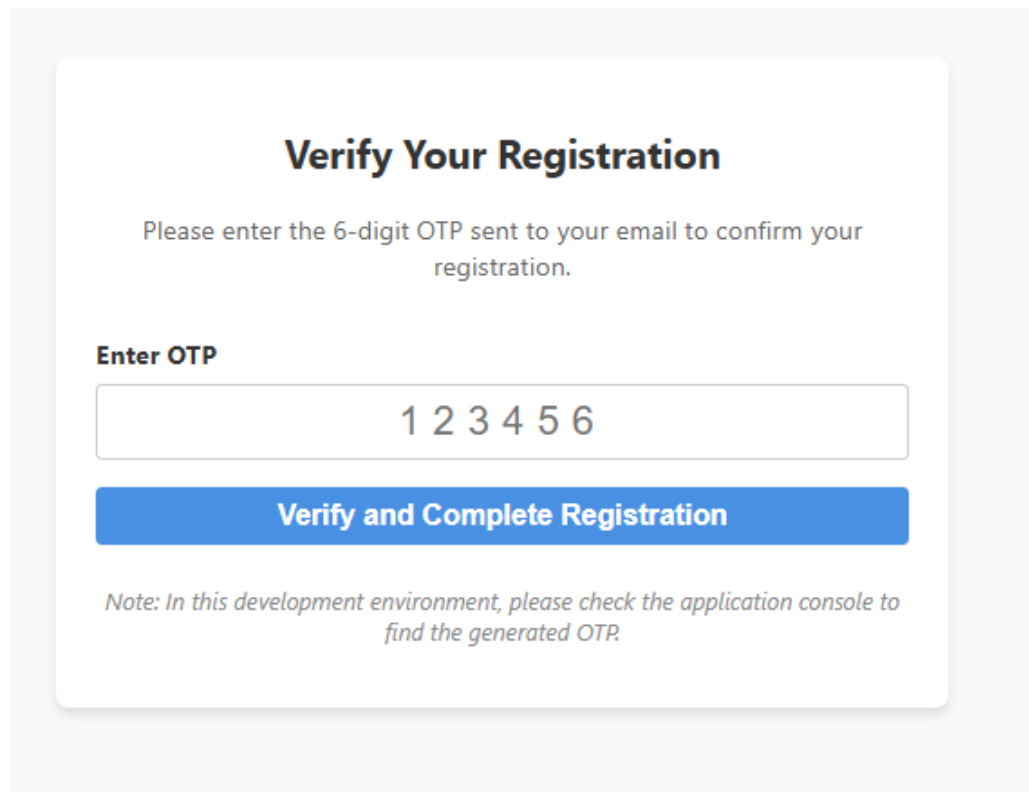
The screenshot displays the 'Manage Events' dashboard. At the top right is a green button labeled 'Add New Event'. Below it is a table with the following data:

Title	Date & Time	Location	Department	Type	Actions
lavaza	2026-05-15 11:40	reasearch park	Computer Science	Seminar	Edit Delete View Registrations
tantraz	2026-05-13 10:30	33 block	Computer Science	Seminar	Edit Delete View Registrations

Figure 5.6: Event Management

From the Fig 5.6, The Event Management Dashboard provides a structured view of all academic and technical events organized across departments. Each event entry includes essential information such as the title, date, type of event, and the responsible department. The dashboard also tracks student participation through the registration count, indicating the number of students registered out of the total available slots. Additionally, the system provides action options such as viewing event details, editing event information, and deleting events, ensuring efficient management and control over event data.

5.3.7 Email Verification Page

The image shows a web form titled "Verify Your Registration". Below the title is a message: "Please enter the 6-digit OTP sent to your email to confirm your registration." There is a label "Enter OTP" above a text input field. The input field contains the numbers "1 2 3 4 5 6". Below the input field is a blue button with the text "Verify and Complete Registration". At the bottom of the form, there is a note: "Note: In this development environment, please check the application console to find the generated OTP." The entire form is centered on a light gray background.

Verify Your Registration

Please enter the 6-digit OTP sent to your email to confirm your registration.

Enter OTP

1 2 3 4 5 6

Verify and Complete Registration

Note: In this development environment, please check the application console to find the generated OTP.

Figure 5.7: Email Verification

From the Fig 5.7, This page is used to verify the user's email during event registration. A 6-digit OTP (One-Time Password) is sent to the user's email address. The user must enter the OTP in the given field to confirm their identity. After entering the correct OTP, the user can click on "Verify & Register" to complete the registration process. There is also a "Cancel Registration" option to stop the registration if needed.

5.3.8 Registration Success Page

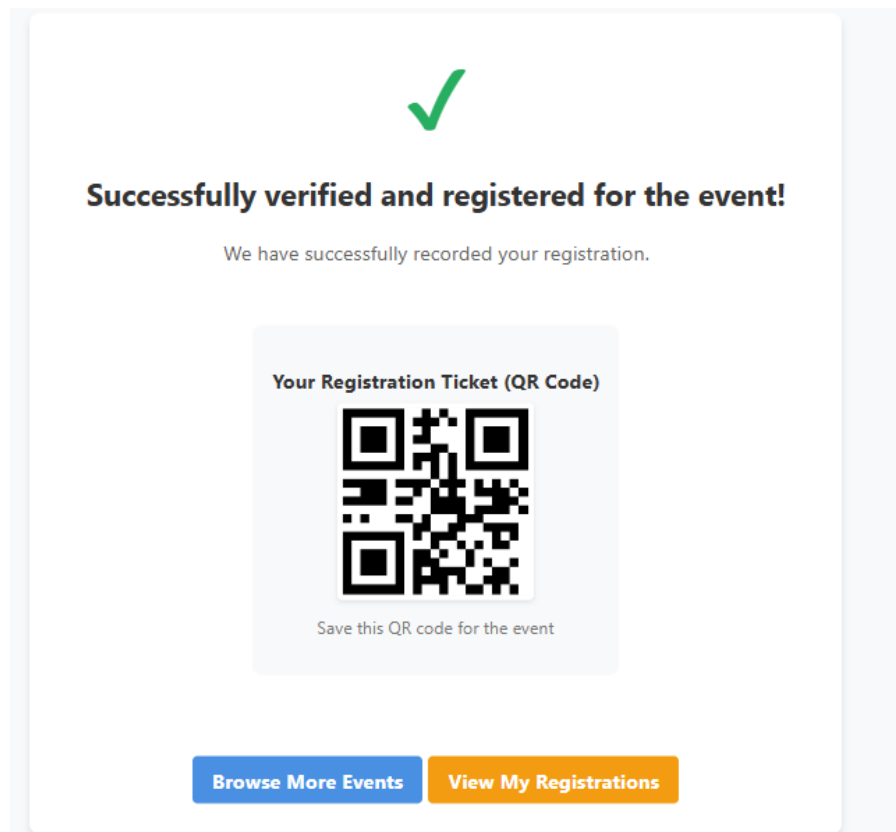


Figure 5.8: Registration Success

From the Fig 5.8, This page shows that the event registration is successful. It confirms that the user's email has been verified and the seat has been booked. It also displays a digital ticket for the event, including the event name, date, and a QR code. The QR code can be used for entry or verification at the event. Additionally, the page contains student details such as name, student ID, and registration ID for identification purposes.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System successfully simplifies the process of managing events in an efficient and organized manner. The system provides features such as event creation, registration, and data management through a user-friendly interface. By using modern technologies like Spring Boot and MySQL, the application ensures reliable performance and easy data handling. It reduces manual effort, minimizes errors, and improves overall productivity.

6.2 Future Enhancements

The system can be further improved by adding advanced features such as email and SMS notifications for event updates. User authentication can be enhanced using secure login methods and role-based access control. The application can be extended to support online event registration with payment integration. It can also be deployed on cloud platforms for wider accessibility. Additionally, a mobile application version can be developed to improve user convenience and accessibility.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3–WK6	Knowledge of Spring Boot, MVC, database design, REST APIs, and frontend technologies.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4–WK6	Balances usability with security and proper validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3–WK5	Involves system design, database schema, and event handling logic.
WP4	Familiarity	Partially new problems	WK4, WK8	Includes real-time updates and user interaction features.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event management.
WP6	Stakeholder Needs	Multiple users	WK5–WK7	Supports students, organizers, and administrators.
WP7	Interdependence	System integration	WK3–WK6	Integrates frontend, backend, and database modules.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Table 7.1 represents the mapping of the project to Complex Engineering Problems (CEP). It highlights how the system involves various attributes such as depth of knowledge, analysis, conflicting requirements, and system integration. The table demonstrates the application of engineering concepts and technical skills required to design and develop a full-stack event management system.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

Table 7.2 represents the mapping of the project to Complex Engineering Activities (CEA). It explains how the project utilizes diverse technologies, supports complex module interactions, and incorporates modern development practices. The table also emphasizes the impact of the system on users and its contribution toward improving efficiency and usability.

7.3 SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

Table 7.3: SDG Mapping for the Project

Table 7.3 represents the mapping of the project to Sustainable Development Goals (SDGs). It shows how the system contributes to quality education, digital transformation, sustainability, and secure management practices. The table highlights the social and environmental relevance of the event management system.

References

- [1] **Eventbrite:** Available at: <https://www.eventbrite.com> Review: Eventbrite is a widely used platform for event creation and ticket management. It provides advanced features such as online booking and promotions but can be complex for small-scale use.

- [2] **Meetup:** Available at: <https://www.meetup.com> Review: Meetup focuses on connecting people with common interests and organizing group events. It is useful for community-based events but limited for institutional management.

- [3] **Google Events:** Available at: <https://calendar.google.com> Review: Google Events integrates with Google Calendar to manage schedules and reminders. It is simple to use but lacks dedicated event management features like registration tracking.

- [4] **Zoho Backstage:** Available at: <https://www.zoho.com/backstage/> Review: Zoho Backstage is an event management platform that offers features like event planning, ticketing, and analytics. It is suitable for professional events but may be complex for beginners and small-scale use.

- [5] **Cvent:** Available at: <https://www.cvent.com> Review: Cvent provides comprehensive event management solutions including registration, venue selection, and reporting. It is widely used by enterprises but can be expensive and not ideal for simple applications.

SOURCE CODE

This chapter presents the key source code snippets of the Smart Campus Event Management System. The project is developed using Spring Boot and follows a layered architecture including Controller, Service, Repository, and Entity layers.

8.1 Main Application Entry Point

Listing 7.1: SmartCampusEventManagerApplication.java – Spring Boot Entry Point

```
1 package com.smartcampus;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class SmartCampusEventManagerApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(SmartCampusEventManagerApplication.class
10             , args);
11     }
```

8.2 Event Entity (Model)

Listing 7.2: Event.java – JPA Entity

```
1 package com.smartcampus.entity;
```

```

2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class Event {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String name;
13    private String description;
14    private String date;
15    private String location;
16
17    // Getters and Setters
18 }

```

8.3 Registration Entity (Model)

Listing 7.3: Registration.java – JPA Entity

```

1 package com.smartcampus.entity;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class Registration {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String studentName;

```

```

13     private String email;
14
15     @ManyToOne
16     @JoinColumn(name = "event_id")
17     private Event event;
18
19     // Getters and Setters
20 }

```

8.4 Event Repository

Listing 7.4: EventRepository.java – Spring Data JPA

```

1 package com.smartcampus.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import com.smartcampus.entity.Event;
5
6 public interface EventRepository extends JpaRepository<Event, Long> {
7 }

```

8.5 Registration Repository

Listing 7.5: RegistrationRepository.java – Spring Data JPA

```

1 package com.smartcampus.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import com.smartcampus.entity.Registration;
5
6 public interface RegistrationRepository extends JpaRepository<
7     Registration, Long> {

```

8.6 Event Service

Listing 7.6: EventService.java – Business Logic

```
1 package com.smartcampus.service;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5 import java.util.List;
6 import com.smartcampus.entity.Event;
7 import com.smartcampus.repository.EventRepository;
8
9 @Service
10 public class EventService {
11
12     @Autowired
13     private EventRepository eventRepository;
14
15     public List<Event> getAllEvents() {
16         return eventRepository.findAll();
17     }
18
19     public Event saveEvent(Event event) {
20         return eventRepository.save(event);
21     }
22
23     public void deleteEvent(Long id) {
24         eventRepository.deleteById(id);
25     }
26 }
```

8.7 Registration Service

Listing 7.7: RegistrationService.java – Business Logic

```

1 package com.smartcampus.service;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5 import com.smartcampus.entity.Registration;
6 import com.smartcampus.repository.RegistrationRepository;
7
8 @Service
9 public class RegistrationService {
10
11     @Autowired
12     private RegistrationRepository registrationRepository;
13
14     public Registration register(Registration registration) {
15         return registrationRepository.save(registration);
16     }
17 }

```

8.8 Event Controller

Listing 7.8: EventController.java – Event Management

```

1 package com.smartcampus.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
6 import org.springframework.web.bind.annotation.*;
7 import com.smartcampus.entity.Event;
8 import com.smartcampus.service.EventService;
9
10 @Controller
11 public class EventController {
12

```

```

13     @Autowired
14     private EventService eventService;
15
16     @GetMapping("/")
17     public String viewEvents(Model model) {
18         model.addAttribute("events", eventService.getAllEvents());
19         return "index";
20     }
21
22     @GetMapping("/admin")
23     public String adminPage(Model model) {
24         model.addAttribute("event", new Event());
25         return "admin";
26     }
27
28     @PostMapping("/save")
29     public String saveEvent(@ModelAttribute Event event) {
30         eventService.saveEvent(event);
31         return "redirect:/";
32     }
33 }

```

8.9 Registration Controller

Listing 7.9: RegistrationController.java – User Registration

```

1 package com.smartcampus.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.web.bind.annotation.*;
6 import com.smartcampus.entity.Registration;
7 import com.smartcampus.service.RegistrationService;
8

```

```

9 @Controller
10 public class RegistrationController {
11
12     @Autowired
13     private RegistrationService registrationService;
14
15     @PostMapping("/register")
16     public String register(@ModelAttribute Registration registration) {
17         registrationService.register(registration);
18         return "success";
19     }
20 }

```

8.10 Security Configuration

Listing 7.10: SecurityConfig.java – Basic Authentication

```

1 package com.smartcampus.config;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.security.config.annotation.web.builders.
    HttpSecurity;
5 import org.springframework.security.web.SecurityFilterChain;
6 import org.springframework.context.annotation.Bean;
7
8 @Configuration
9 public class SecurityConfig {
10
11     @Bean
12     public SecurityFilterChain filterChain(HttpSecurity http) throws
        Exception {
13         http
14             .authorizeHttpRequests(auth -> auth
15                 .requestMatchers("/admin/**").authenticated()

```

```

16         .anyRequest().permitAll()
17     )
18     .formLogin();
19
20     return http.build();
21 }
22 }

```

8.11 Application Properties

Listing 7.11: application.properties – Configuration

```

1 spring.datasource.url=jdbc:mysql://localhost:3306/campus_db
2 spring.datasource.username=root
3 spring.datasource.password=1234
4
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7
8 spring.thymeleaf.cache=false

```

8.12 Sample Frontend Page

Listing 7.12: index.html – Event Listing Page

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Campus Events</title>
5 </head>
6 <body>
7     <h1>Campus Events</h1>
8
9     <div th:each="event, _" ${events}">

```

```
10         <h3 th:text="{event.name}"></h3>
11         <p th:text="{event.description}"></p>
12     </div>
13 </body>
14 </html>
```